ECE 2230: Data Structures and Algorithms for ECE **Fall 2026**
Machine Problem 3 **Due: 11:59 pm, Monday, Oct 26**

Weight: 15% of the course grade.

The goal of this machine problem is to extend the two-way linked list ADT from machine problem 2 to include five algorithms for sorting a list. Extend machine problem 2 to include the new commands:

`SORTGEN` *x*
`SORTIP` *x*
`APPENDREAR generator_id dest_ip_addr`

`SORTGEN` *x* sorts the queue in ascending order based on **`generator_id`**. **`SORTIP`** *x* sorts the queue in decending order based on **`dest_ip_addr`**. For both sorting methods, *x* is one of the sorting algorithms defined below.

`APPENDREAR` is similar to **`ADDREAR`** except for the following changes. First, unlike **`ADDREAR`**, do NOT check if the destination IP address is already found in the unsorted list. Simply add the new alert record to the back of the queue. This implies that there CAN be duplicate **`generator_id`** and **`dest_ip_addr`**'s. Second, both the **`generator_id`** and **`dest_ip_addr`** are given on the command line. The remaining parameters in the **`alert _t`** structure are all set to zero by default. Don't call the **`ids_record_fill`** function. Just collect the **`generator_id`** and **`dest_ip_addr`** (and use **`calloc`** so all other values in the structure are zero). Later in the semester we will discover tools that can make searches for duplicates more efficient. For this MP we will skip that detail.

The code must consist of the following files:

`lab3.c`	– the `main()` function for testing the sort algorithms. Use **`lab2.c`** as a template.
`llist.c`	– Extend the two-way linked list ADT from lab2
`ids_support.c`	– Extend the **`ids_support.c`** file from machine problem 2.
`ids_support.h`	– The IDS interface definitions.
`llist.h`	– The data structures and prototype definitions for the list ADT.
`datatypes.h`	– Key definitions of the alter structure
`makefile`	– Compiler commands for all code files

Sorting the two-way linked list ADT

Create the prototype definition `void llist_sort(llist_t *list_ptr, int sort_type, int (*fcomp)(const data_t *, const data_t *))` in your `llist.h` file. For this function, `sort_type` can take one of the following values:

1. Insertion sort. We can use the other functions in `llist.c` to implement a very simple sort function. To sort the list, use `llist_remove` to take the first (or front) item from the list and `llist_insert_sorted` to put the item into a second list that is sorted. When all the elements have been inserted into the sorted list, adjust the pointers in `list_ptr` to point to the newly sorted list. Note this is a simple variation of the priority queue sort described by Standish in Section 4.3 of the book.
2. Recursive Selection Sort. Implement the recursive version of the selection sort as defined by program 5.19 on page 152 in the book by Standish. Update the algorithm so that it properly handles our two-way linked list (as opposed to the implementation for an array in program 5.19). The calculations of

the sort algorithm should not change; just change the algorithm to work with pointers instead of indexes into an array. You will also need to implement Standish's `FindMax` algorithm defined in program 5.20 on page 152. The source code from Standish is included in the starter ZIP on canvas in **`StandishSort.c`**; use this code to get a quick start on both selection sort algorithms. For an important hint, see also the note at the end of this document. Note it is critical that you don't change the logic of the algorithm. Simply update the algorithm to use pointers instead of integer indices.

3. Iterative Selection Sort. Implement the iterative version of the selection sort as defined by program 5.35 on page 171 in the book by Standish. Read Section 5.4 for an explanation of how the recursive version of the program is transformed into the iterative version.
4. Merge Sort. Implement the recursive version of merge sort as defined by the program 6.19 on page 237 of the book by Standish. Note that you will need to implement two support functions. The first is a function to partition a list into two half-lists (this is easy to implement as you just step through the linked list until half the list size and then break the list into two lists). This second function merges two lists that are sorted into a single list. Read the paragraph on page 237 that discusses how to merge two lists, and note that it is a simple process of using `llist_remove` at the front of either the left or right list and `llist_insert` at the back of the merged list. See also the MergeSort powerpoint file for an illustration of the algorithm in the textbook.
5. Quick Sort. The C standard library includes a highly optimized implementation of Quick Sort called **`qsort`**. However, it is designed to work with an array only. At the end of this PDF file is an example of how to set up the call to the **`qsort_r`** library function. The code is given to you in the **`StandishSort.c`** file.

Be careful to design your algorithms so that you do not change their complexity class. Do not redesign the algorithms! The final two lines of the function `llist_sort` **must** be

```
list_ptr->ll_sorted_state = LLIST_SORTED;
llist_debug_validate(list_ptr);
```

The **`llist_sort()`** function must also update the **`compare_fun`** structure member in the list header to be equal to the function pointer **`fcomp`** (i.e., **`list_ptr->compare_fun = fcomp`**). Note that **`APPENDREAR`** is implemented using **`llist_insert()`**, and the **`llist_insert`** function changes **`ll_sorted_state`** to LLIST_UNSORTED. Thus, the sort commands are used the change the list status to the sorted state and set the comparison function pointer to match the type of information that is sorted.

Measuring time to sort

To measure the performance of a sorting algorithm use the built in C function **`clock`** to count the number of cycles used by the program. In **`ids_support.c`** add two functions for sorting and use code similar to this:

```
#include <time.h>
clock_t start, end;
double elapse_time;  /* time in milliseconds */

int initialcount = llist_entries(L);
start = clock();
llist_sort(L, sort_type, ids_comparison_function);
end = clock();
elapse_time =  1000.0 * ((double) (end - start)) / CLOCKS_PER_SEC;
assert(llist_entries(L) == initialcount);
printf("%d\t%f\t%d\n", initialcount, elapse_time, sort_type);
```

where `CLOCKS_PER_SEC` and `clock_t` are defined in `<time.h>`.

Additional requirements for SORT

Your final code must use the exact `printf()` statement given in the above example. You will collect output from multiple runs to plot performance curves, showing run time for various list sizes. Your program **must** verify that the size of the list after the completion of the call to `llist_sort` matches the size before the list is sorted. Your program must not have any memory leaks or array boundary violations. The format of the print makes it easy to import the data into a spreadsheet program.

Suppress prints and unnecessary validation calls during performance evaluation and for final submission

Do not include any **`printf`** calls for your new **`ids_append_rear`** function. We don't need to see 250,000 prints about prompting and adding to the queue. Also, comment out the welcome menu, print for line that is not a command (the print with the #), and goodbye prints in **`lab3.c`** to avoid unnecessary chatter. The **only** output should be the one new print statement added to measure the sorting time. Make sure to use **`calloc`** so that **`valgrind`** does not report uninitialized values. (Instead of using the function to collect alert information, you will instead just use a **`calloc`** and set the **`generator_id`** and **`dest_ip_addr`**.)

The `llist_debug_validate()` function is **very** inefficient. After all your code works correctly, remove `llist_debug_validate()` from **all** `llist_()` functions except `llist_sort()`, where it must remain as the last line called before the function returns.

Generating large inputs for testing

See the supplemental program **`geninput.c`** to create input for testing. The program takes four options on the command line. The first specifies the size of the list, the second specifies the type of list, the third the type of sorting algorithm, and the forth if the sorting should be based on the generator ID (`gen`) or the destination IP address (`ip`). There are three possible types:

1. List with elements in a random order.
2. List with elements already in ascending order.
3. List with elements in descending order.

Run `./geninput` with no arguments to print a help message. To run, pipe the output of the `geninput` program into `lab3`. For example, for a merge sort trial on a list with 10,000 elements in random order use:

```
./geninput 10000 1 4 gen | ./lab3
```

The final code you submit **must** operate with **`geninput.c`**. In particular, you must have commented out the calls to the validation function (except the call in **`llist_sort`**). Your program should be able to sort approximately 12,000 items (or more) in about one second for insertion sort or either selection sort. For merge sort, your program should be able to sort approximately 500,000 items (or more) in about one second

Final testing and PDF for the testing log

Test each of the sorting algorithms and each of the three list types (random, ascending, descending) with at least **five** different list sizes. All performance evaluation is run using the generator ID as the sort type

(there should be no significant different if the destination IP address is used to sort). You **must** create graphs to illustrate your results. In particular, for the tests involving random list types, you **must** include in your graph the result for at least **one** list size that requires more than one second to sort as determined using the C function `clock`. See the file **`mp3test.sh`** for a template that you can edit.

In your Test Log document, in addition to reporting your data using graphs, describe

(a) For lists that are initially random, explain the differences in running time for the sorting algorithms. Do your iterative and recursive selection sort algorithms show dramatic differences in running times or are they similar? Why does the mergesort algorithm show a dramatic improvement in run time? If the runtime for merge sort is not dramatically faster than the other algorithms you have a bug.

(b) If a list is already in ascending or descending order, some sort algorithms are very fast while others still have to perform a similar number of comparisons as when the list is not sorted. Describe which algorithm(s) show extremely fast performance if the list is already sorted, and explain why.

You must submit your test log as a PDF file. Both Microsoft Word and LibreOffice have tools that convert the document to PDF format.

Your test script is a simple listing of the lab3 commands you used to generate the data for your test log. See the example script **`mp3test.sh`** that is part of the starter ZIP on Canvas. You just need to make small changes to the test script to account for the speed of your computer.

An optional experiment is to recompile your final code taking out the –Wall and –g options and adding the –O option (capital letter o, not zero) to all calls with gcc. The –O option turns on compiler optimizations and you should find your code runs faster. While the run times are reduced and you can sort larger lists, has the complexity class for any of the sorting algorithms changed?

Hint

When converting Standish's selection sort algorithms from working with arrays to working with pointers, don't try to add array-like features to our two-way linked list. Instead rewrite Standish's code to use pointers. For example, change

```
void SelectionSort(InputArray A, int m, int n)
```

to

```
void SelectionSort(llist_t *A, llist_elem_t *m, llist_elem_t *n)
```

The changes you need to make should be very simple. For example, change from incrementing index values to moving roving pointers.

The **`qsort`** library function requires the items to be stored in an array and there must be a comparison function. One simple approach is to store the pointers to our **`data_t`** blocks in array (but do not use arrays for any of the other sorting algorithms!). However, **`qsort`** then must dereference a pointer twice to compare **`data_t`** blocks. The first pointer indexes into the array, the second pointer is the address of the **`data_t`** memory block. Note the **`qsort`** function swaps the pointers, not the **`data_t`** memory blocks. Here is the code you can use in **`llist.c`** (or see the code in the file **`StandishSort.c`**):

You must add this define as the first line in **`llist.c`** (after the initial comments):

```
#define _GNU_SOURCE   // so that stdlib.h defines qsort_r
```

Then in your **`llist_sort`** function for the qsort option:

```
int i, Asize = llist_entries(list_ptr);
assert(list_ptr->compare_fun != NULL);
data_t ** QsortA = (data_t **) malloc(Asize*sizeof(data_t *));
for (i = 0; i < Asize; i++) {
    QsortA[i] = llist_remove(list_ptr, LLPOSITION_FRONT);
}
qsort_r(QsortA, Asize, sizeof(data_t *), my_compare, list_ptr);
for (i = 0; i < Asize; i++) {
    llist_insert(list_ptr, QsortA[i], LLPOSITION_BACK);
}
free(QsortA);
```

/* Comparison function for qsort */

```
int my_compare(const void *p_a, const void *p_b, void * lptr)
{
    llist_t *list_ptr = (llist_t *) lptr;
    return list_ptr->compare_fun (*(data_t **) p_b, *(data_t **) p_a);
}
```

Submission

Submit all the files in one ZIP file to the MP3 assignment on Canvas. Be sure to include all the files listed on page 1, your test script, and a PDF document that includes graphs showing performance of all sorting algorithm. See the README file for Canvas submission instructions. You can make more than one submission but we will only grade the final submission.

To receive credit for this assignment your code must compile and at a minimum sort small lists using all sorting algorithms. Code that does not compile or does not sort sets of 100 records will not be accepted or graded. Merge sort must be able to handle list sizes of at least 1,000,000 items.

See the ECE 2230 Programming Guide for additional requirements that apply to all programming assignments.

Work must be completed by each individual student, and see the course syllabus for additional policies. The work you submit should be your own words and computer code that you wrote for this assignment.